Design Document for N3RD M4RKET

Group <4_az_1>

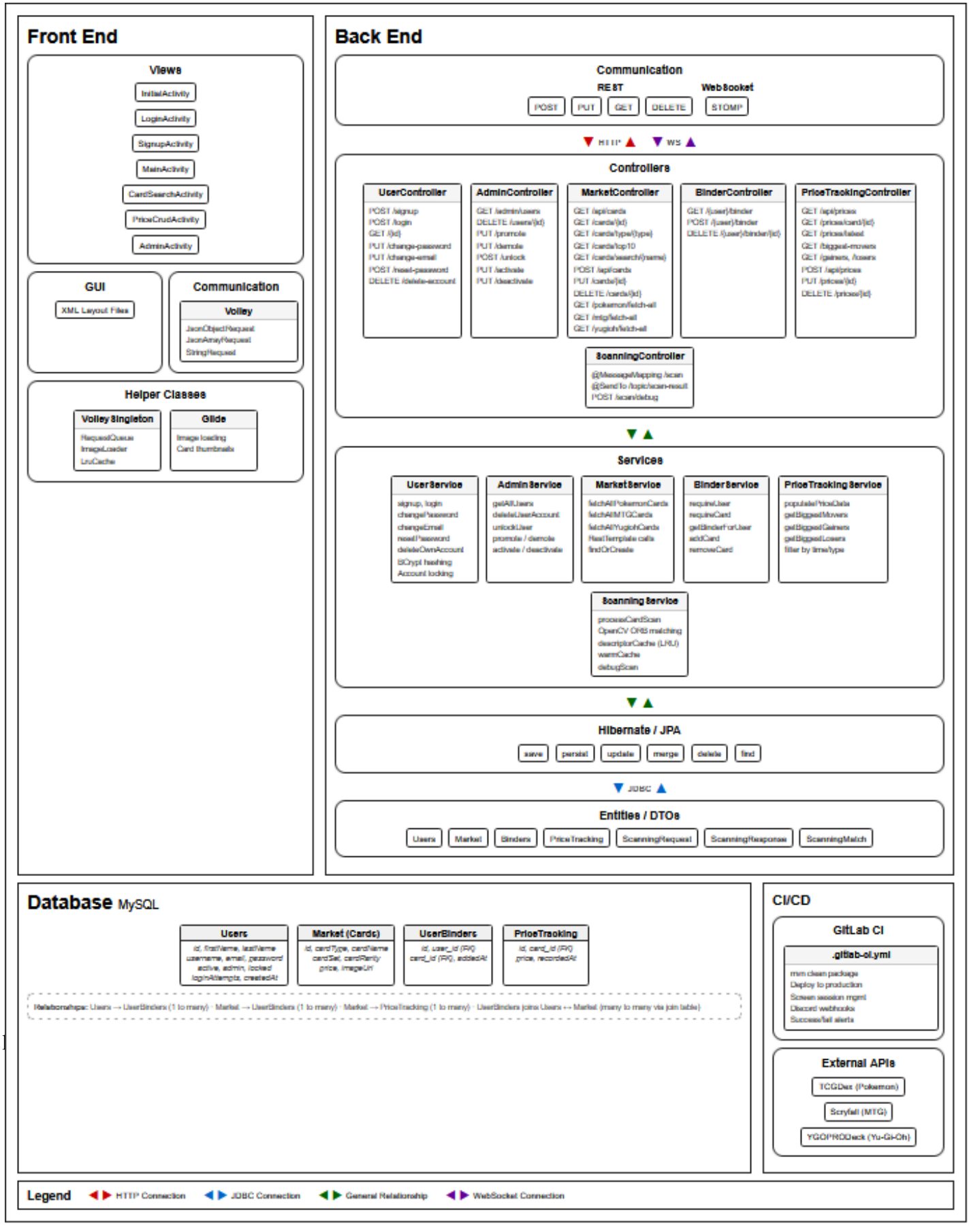
Hrushi Bhatt: 1000 % contribution

Cooper Hoy: 2000% contribution

Logan Heileman: 3000% contribution

Rudra Naik: 4000% contribution

Backend SwaggerUI: http://coms-3090-022.class.las.iastate.edu:8080/swagger-ui/index.html


Front End
Views
InitialActivity
LoginActivity
SignupActivity
MainActivity
CardSearchActivity
PriceCrudActivity
AdminActivity
GUI
XML Layout Files
Communication
Volley
JsonObjectRequest
JsonArrayRequest
StringRequest
Helper Classes
Volley Singleton
RequestQueue
ImageLoader
LruCache
Glide
Image loading
Card thumbnails
Back End
Communication
REST
POST
PUT
GET
DELETE
WebSocket
STOMP
▼ HTTP ▲ ▼ WS ▲
Controllers
UserController
POST /signup
POST /login
GET /{id}
PUT /change-password
PUT /change-email
POST /reset-password
DELETE /delete-account
AdminController
GET /admin/users
DELETE /users/{id}
PUT /promote
PUT /demote
POST /unlock
PUT /activate
PUT /deactivate
MarketController
GET /api/cards
GET /cards/{id}
GET /cards/type/{type}
GET /cards/top10
GET /cards/search/{name}
POST /api/cards
PUT /cards/{id}
DELETE /cards/{id}
GET /pokemon/fetch-all
GET /mtg/fetch-all
GET /yugioh/fetch-all
BinderController
GET /{user}/binder
POST /{user}/binder
DELETE /{user}/binder/{id}
PriceTrackingController
GET /api/prices
GET /prices/card/{id}
GET /prices/latest
GET /biggest-movers
GET /gainers, /losers
POST /api/prices
PUT /prices/{id}
DELETE /prices/{id}
ScanningController
@MessageMapping /scan
@SendTo /topic/scan-result
POST /scan/debug
▼ ▲
Services
UserService
signup, login
changePassword
changeEmail
resetPassword
deleteOwnAccount
BCrypt hashing
Account locking
AdminService
getAllUsers
deleteUserAccount
unlockUser
promote / demote
activate / deactivate
MarketService
fetchAllPokemonCards
fetchAllMTGCards
fetchAllYugiohCards
RestTemplate calls
findOrCreate
BinderService
requireUser
requireCard
getBinderForUser
addCard
removeCard
PriceTracking Service
populateFromCards
getBiggestMovers
getBiggestGainers
getBiggestLosers
filter by time/type
Scanning Service
processCardScan
OpenCV ORB matching
descriptorCache (LRU)
warmCache
debugScan
▼ ▲
Hibernate / JPA
save
persist
update
merge
delete
find
▼ JDBC ▲
Entities / DTOs
Users
Market
Binders
PriceTracking
ScanningRequest
ScanningResponse
ScanningMatch
Database MySQL
Users
id, firstName, lastName
username, email, password
active, admin, locked
loginAttempts, createdAt
Market (Cards)
id, cardType, cardName
cardSet, cardRarity
price, imageUrl
UserBinders
id, user_id (FK)
card_id (FK), addedAt
PriceTracking
id, card_id (FK)
price, recordedAt
Relationships: Users → UserBinders (1 to many) · Market → UserBinders (1 to many) · Market → PriceTracking (1 to many) · UserBinders joins Users ↔ Market (many to many via join table)
CI/CD
GitLab CI
.gitlab-ci.yml
mvn clean package
Deploy to production
Screen session mgmt
Discord webhooks
Success/fail alerts
External APIs
TCGDex (Pokemon)
Scryfall (MTG)
YGOPRODeck (Yu-Gi-Oh)
Legend
HTTP Connection
JDBC Connection
General Relationship
WebSocket Connection

Frontend

Communication: The frontend used Android Studio as the IDE. Communication with the backend is achieved through Volleys for HTTP requests using JSON bodies for requests and responses. These HTTP requests support all CRUD functionality across different classes. For websocket implementation, STOMP is used for live notifications and card scanning via camera.

Classes: The frontend is divided into many classes that handle the UI and communication with the backend. These include:

- **AdminActivity:** Handles all admin related functions such as deleting users, unlocking users, promoting and demoting specific users statuses, unlocking and locking compromised accounts, deleting accounts, and then searching for specific accounts as needed. Is only accessible to users promoted to admin.
- **CardSearchActivity:** Allows the user to search for specific cards by their names, along with allowing them to see a candlestick graph for the price fluctuation. Also lets users add and remove cards from their binder.
- **CardBinderActivity:** Allows users to look at the cards that are in their binder, along with removing them, and also being able to look at more detailed portfolio information, and search for specific cards that they may have.
- **InitialActivity:** Basic splash screen the user is presented with before being redirected to the login screen.
- **LoginActivity:** Handles username and password by sending to the backend via volleys. If valid, user ID/username is stored and they are logged in to their account and presented with the main screen.
- **MainActivity:** Presents users with their individualized portfolio as well as being a hub for navigation to different pages such as the card search, biggest movers, and admin view.
- **PriceCrudActivity:** Allows for an admin to edit the price logs of any card in the database if something must be changed. Enables them to add a new entry, edit an existing one, see all entries, or delete an entry.
- **SignupActivity:** Prompts the user for username and password, as well as other optional fields such as name and email. Parses the inputted password to make sure it meets requirements (8+ characters, 1 or more special characters) and sends via POST to the backend if valid.
- **CameraSearchActivity:** Enables full camera functionality and prompts users to take a photo of a card. Uses Tesseract OCR to identify the card name and sends that as well as the picture of the card to the backend to search for matches.

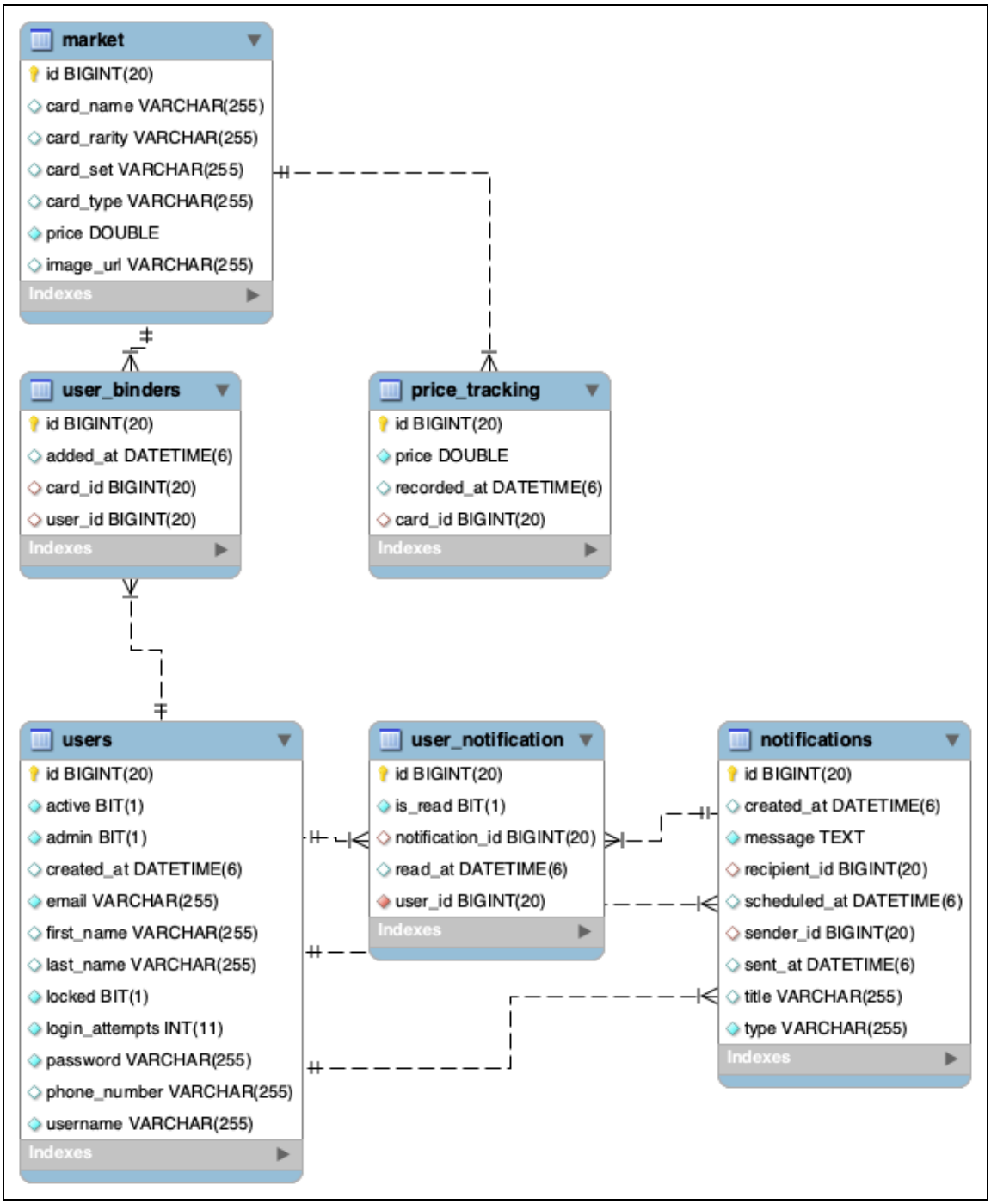
Backend

Communication: The backend uses Spring Boot with Apache Tomcat as the embedded server. All communication between the Android frontend and the server is handled through RESTful API mappings using JSON request and response bodies. The server exposes endpoints organized by feature, supporting standard HTTP methods: POST for creating resources, GET for retrieving data, PUT for updating existing records, and DELETE for removing entries. In addition to REST, the server uses WebSocket connections via STOMP for real-time push notifications to connected clients.

Controllers: The controllers contain the endpoint mappings that handle communication between the frontend and the database. These include:

- **Market**: Contains CRUD mappings for trading cards. Cards are categorized by type (POKEMON, MTG, YUGIOH) and include fields for name, set, rarity, price, and image URL. Also exposes endpoints to fetch and populate card data from external APIs (TCGdex, Scryfall, YGOPRODeck).
- **User**: Handles user registration (signup), login with password validation, password and email changes, password reset, and account deletion. Passwords are hashed using BCrypt. Accounts lock after repeated failed login attempts.
- **Admin**: Provides administrative endpoints for promoting and demoting users to admin status, activating and deactivating accounts, and unlocking locked accounts. Admin actions require a valid admin userId passed as a query parameter.
- **Notification**: Manages user notifications with endpoints to create notifications, retrieve notifications by user, mark notifications as read, and get unread notification counts. Delivered in real-time through WebSocket when users are connected.
- **Price Tracking:** Records daily price snapshots for cards and exposes endpoints for price history by card, biggest movers across all card types, biggest movers filtered by card type, and biggest movers within configurable time windows (21 days, 7 days, 2 days). Supports filtering by gainers and losers.
- **Binder**: Allows users to save cards to their personal binder (portfolio), retrieve their saved cards, and remove cards from their binder.
- **Scanning**: Allows users to take pictures of their cards in real life and parse the image through our database of cards to search up their card on our application.

Tables and Fields


market
id BIGINT(20)
card_name VARCHAR(255)
card_rarity VARCHAR(255)
card_set VARCHAR(255)
card_type VARCHAR(255)
price DOUBLE
image_url VARCHAR(255)
Indexes
user_binders
id BIGINT(20)
added_at DATETIME(6)
card_id BIGINT(20)
user_id BIGINT(20)
Indexes
price_tracking
id BIGINT(20)
price DOUBLE
recorded_at DATETIME(6)
card_id BIGINT(20)
Indexes
users
id BIGINT(20)
active BIT(1)
admin BIT(1)
created_at DATETIME(6)
email VARCHAR(255)
first_name VARCHAR(255)
last_name VARCHAR(255)
locked BIT(1)
login_attempts INT(11)
password VARCHAR(255)
phone_number VARCHAR(255)
username VARCHAR(255)
Indexes
user_notification
id BIGINT(20)
is_read BIT(1)
notification_id BIGINT(20)
read_at DATETIME(6)
user_id BIGINT(20)
Indexes
notifications
id BIGINT(20)
created_at DATETIME(6)
message TEXT
recipient_id BIGINT(20)
scheduled_at DATETIME(6)
sender_id BIGINT(20)
sent_at DATETIME(6)
title VARCHAR(255)
type VARCHAR(255)
Indexes